SKEL 4223: Digital Signal Processing *with* MATLAB

# CHAPTER 8

# DISCRETE-TIME FOURIER TRANSFORM

Dr. Musa Mohd Mokji

# Content

# 1.0 Plotting Frequency Response

# 1.1 Introduction

Frequency response of a discrete-time signal can be obtained by taking the discrete-time Fourier transform (DTFT) of the signal. However, only discrete Fourier transform (DFT) is available, which is implemented through fast Fourier transform (FFT). DTFT is not available since the frequency $\omega$ (in radian) is a continuous value. Thus, the discrete frequency $k$ in FFT need to be mapped to $\omega$ in order to get the DTFT.

For example, lets plot the DTFT for the following signal $x(t)$:

$$x(t) = sin(2\pi(1000)t) \quad for \quad 0 \leq \ t \leq 1s$$

With its frequency sampling, $F_s = 10kHz$.

## 1.2 Time-Domain Sampling

To convert the continuous-time signal $x(t)$ into discrete-time signal $x[n]$, apply sampling where $t = n/F_s$ or $n = tF_s$. In this example, the 1 second duration of $x(t)$ is sampled into 1001 samples of x[n].

```
Fs = 10000;                   % Frequency sampling
n = 0:1*Fs;                   % sampling 0<=t<=1s to 0<=n<=10000
x = sin(2*pi*1000*n/Fs);      % sampling the signal x(t)
```

## 1.3 Frequency Response

To obtain the frequency response, first, use fft() function to get the DFT, $X[k]$ of the signal $x[n]$. Then, the magnitude response, $|X[k]|$ and phase response, $\angle X[k]$ of the frequency response are obtain using abs() and angle() functions respectively.

```
X = fft(x);
Xmag = abs(X);
Xphase = angle(X);
```

To obtain the DTFT, we need to convert the discrete frequency $k$ to the discrete-time angular frequency, $\omega$ in radian such that $\omega = 2\pi k/N$ where $N$ is the length of $X[k]$. Note that we only converting the x-axis values of the frequency response and do nothing to the y-axis.

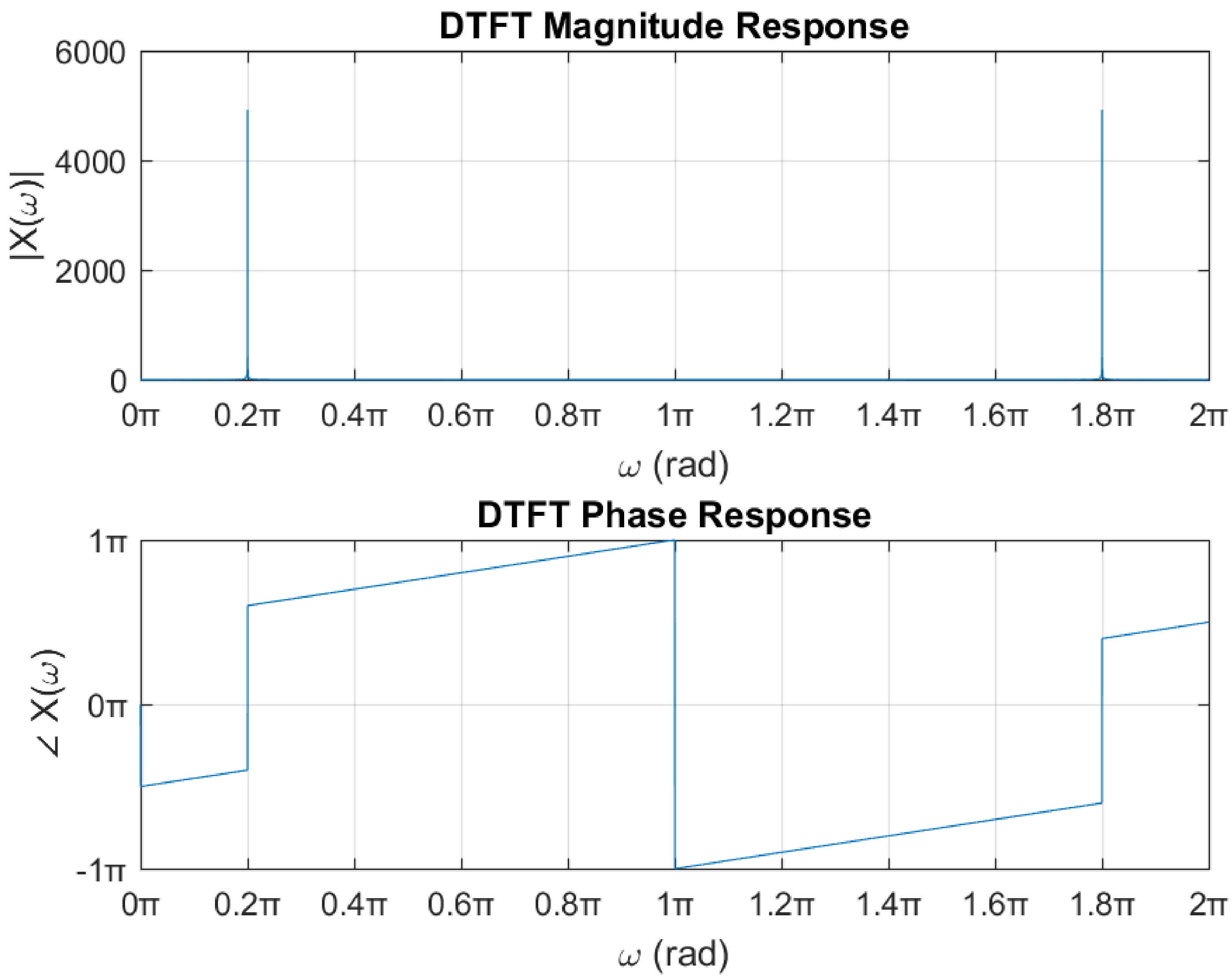
Now, set the discrete frequency $k$ values such that $0 \leq k \leq N-1$ where $k$ is integer values. By default, fft() set $N$, the length of $X[k]$ equals to the the length of $x[n]$. Below is the code to convert the $k$ to $\omega$:

```
N = length(X);
k = 0:N-1;
w = 2*pi*k/N;
```

## 1.4 Plotting the DTFT

Below is how the magnitude and phase responses of the DTFT are plotted on a single window. The frequency $\omega$ is divided by $\pi$ for easier reading the values.

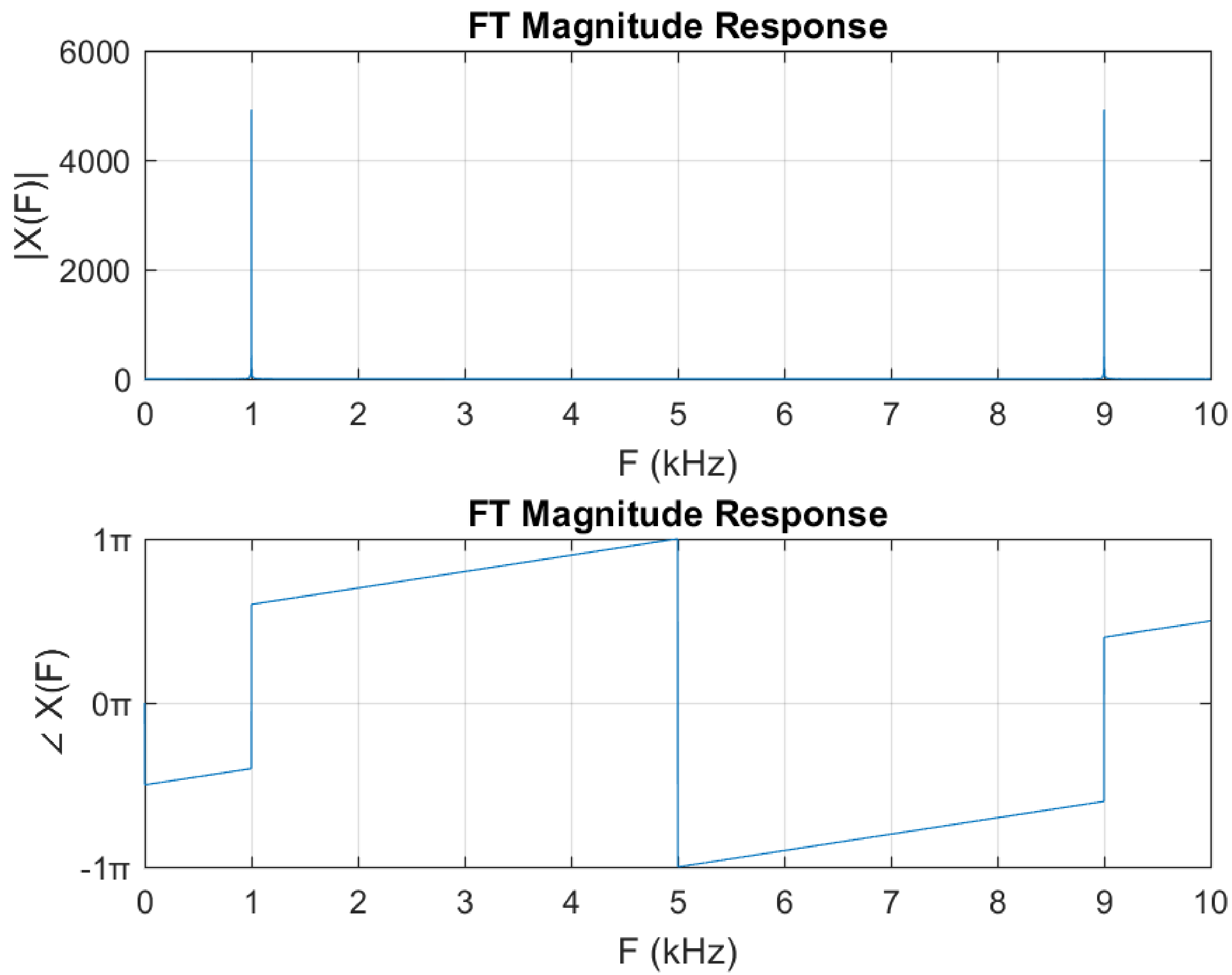
```
subplot(2,1,1), plot(w/pi,Xmag)
xlabel('\omega (rad)'), ylabel('|X(\omega)|')
xticks(0:0.2:2), xtickformat('%g\x3C0')
title('DTFT Magnitude Response')
grid on
subplot(2,1,2), plot(w/pi,Xphase/pi)
xlabel('\omega (rad)'), ylabel('\angle X(\omega)')
xticks(0:0.2:2), xtickformat('%g\x3C0')
yticks(-1:1), ytickformat('%d\x3C0')
title('DTFT Phase Response')
grid on
```


DTFT Magnitude Response
6000
4000
2000
0
|X(ω)|
0π
0.2π
0.4π
0.6π
0.8π
1π
1.2π
1.4π
1.6π
1.8π
2π
ω (rad)
DTFT Phase Response
1π
0π
-1π
∠ X(ω)
0π
0.2π
0.4π
0.6π
0.8π
1π
1.2π
1.4π
1.6π
1.8π
2π
ω (rad)

# 1.5 Plotting Continuous-time Fourier Transform

To display the continuous-time Fourier transform $X(F)$, convert the discrete frequency $k$ into continuous-time frequency $F$ where $F = F_s k/N$. Note that the unit of $F$ is $Hz$.

```
F = Fs*k/N;
figure,
subplot(2,1,1), plot(F/1000,Xmag)
xlabel('F (kHz)'), ylabel('|X(F)|')
xticks(0:10)
title('FT Magnitude Response')
grid on
subplot(2,1,2), plot(F/1000,Xphase/pi)
xlabel('F (kHz)'), ylabel('\angle X(F)')
xticks(0:10)
yticks(-1:1), ytickformat('%d\x3C0')
title('FT Magnitude Response')
grid on
```


FT Magnitude Response
6000
4000
2000
0
|X(F)|
0
1
2
3
4
5
6
7
8
9
10
F (kHz)
FT Magnitude Response
1π
0π
-1π
∠ X(F)
0
1
2
3
4
5
6
7
8
9
10
F (kHz)

# 1.6 Single-Sided Spectrum

Plotting the frequency response for FT such the previous is not accurate since after $F = 5\ kHz$ the plot is supposed to be a mirror to the plot for frequency $0 \leq F \leq 5\ kHz$, which supposed to have a frequency between $-5\ kHz \leq F \leq 0$. Thus, the previous plot is only half correct. To solve this, we either plot single-sided or apply frequency shifting.

Single-sided spectrum is where only half length of the DFT will be considered. The half-length of the DFT can be computed as below:

Even length $N$:

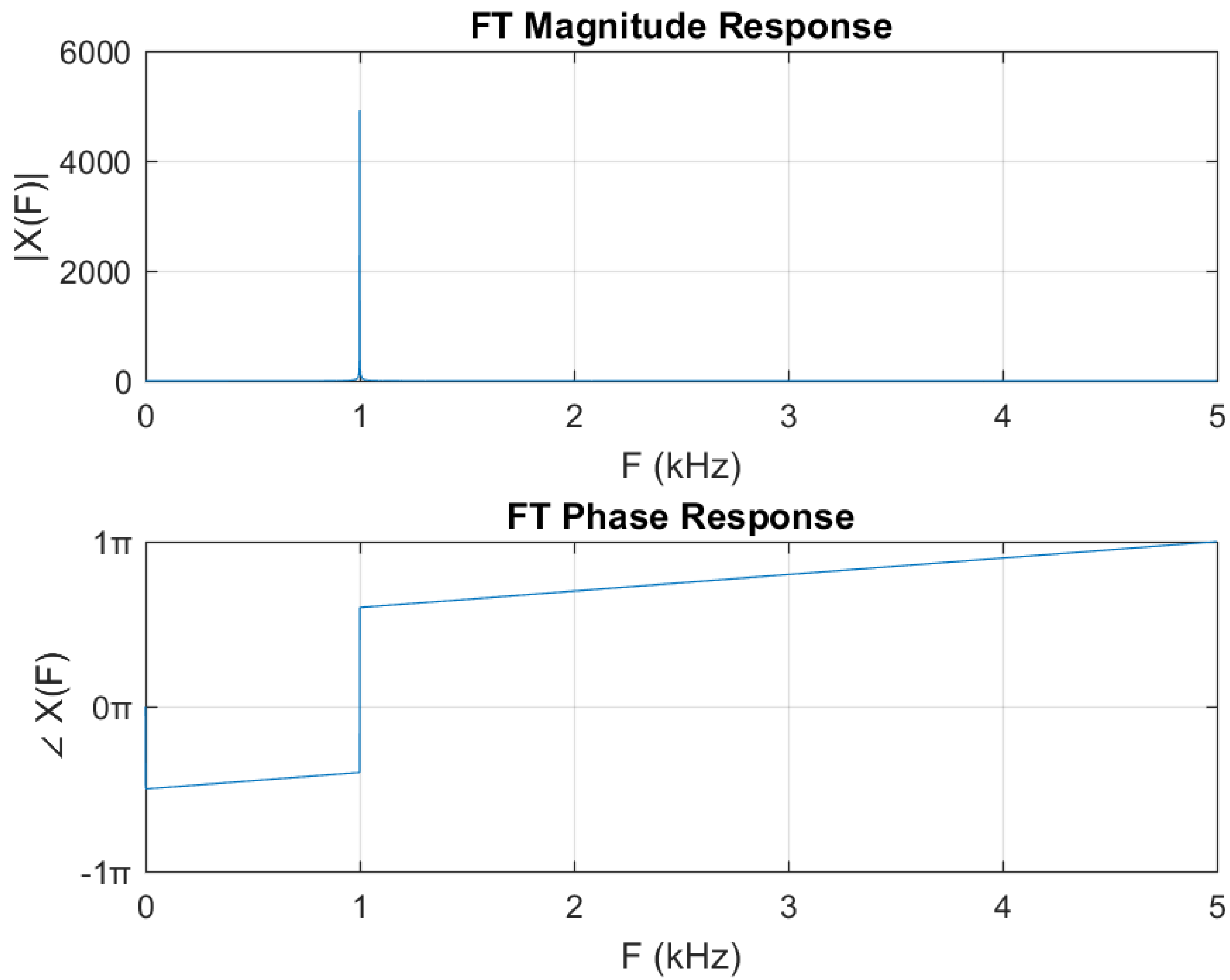
$$N2 = (N + 2)/2$$

Odd length $N$:

$$N2 = (N + 1)/2$$

Below is the code to plot the single-sided spectrum. In this case, $N$ is odd where $N = 1001$.

```
N2 = (N+1)/2;

figure,
subplot(2,1,1), plot(F(1:N2)/1000,Xmag(1:N2))
xlabel('F (kHz)'), ylabel('|X(F)|')
title('FT Magnitude Response')
grid on
subplot(2,1,2), plot(F(1:N2)/1000,Xphase(1:N2)/pi)
xlabel('F (kHz)'), ylabel('\angle X(F)')
ylim([-1 1]), yticks(-1:1), ytickformat('%g\x3C0')
title('FT Phase Response')
grid on
```


FT Magnitude Response
|X(F)|
6000
4000
2000
0
0
1
2
3
4
5
F (kHz)
FT Phase Response
∠ X(F)
1π
0π
-1π
0
1
2
3
4
5
F (kHz)

# 1.7 Frequency Shifted Spectrum

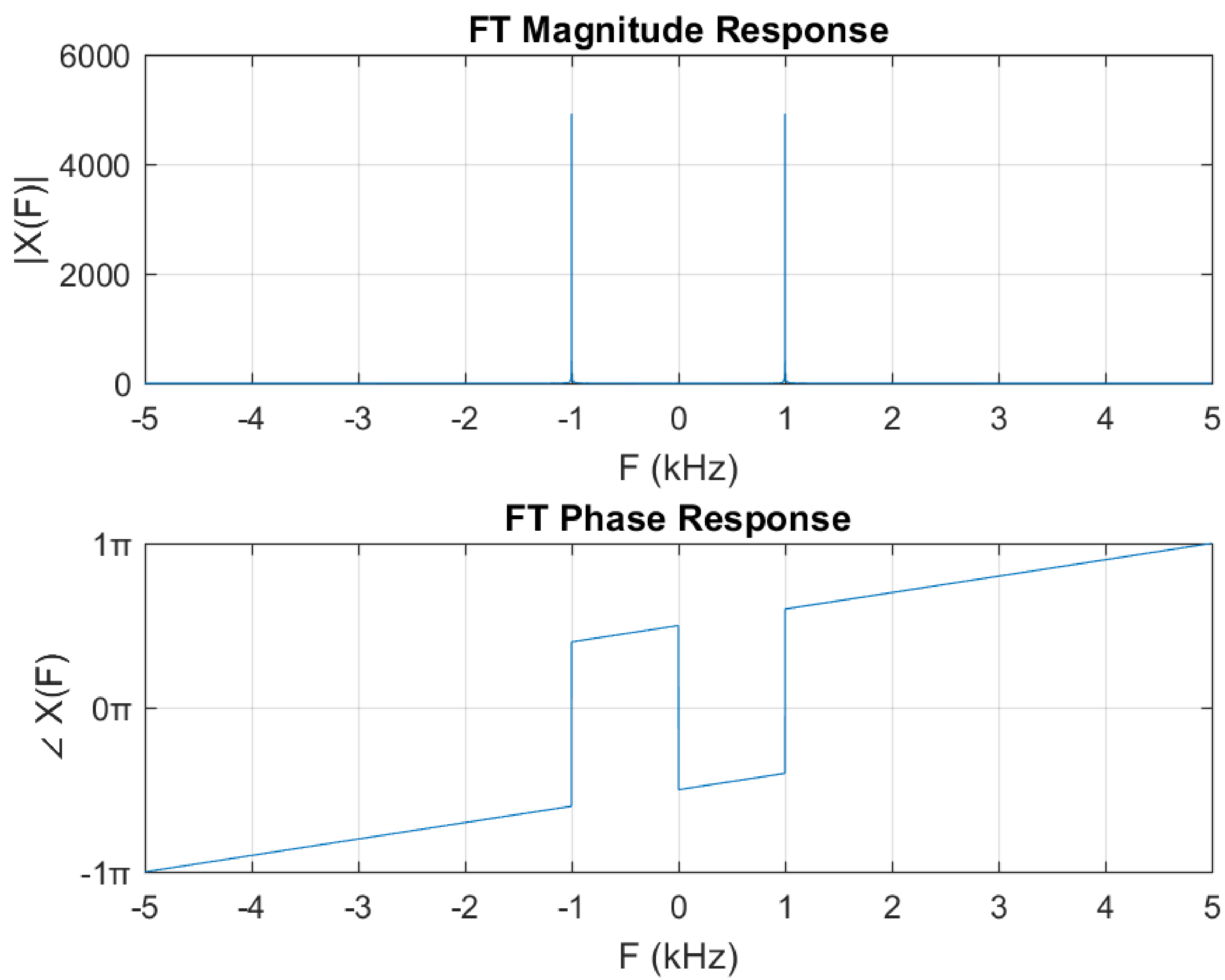
In frequency shifting method, the frequency response is plotted with range of $-F_m \leq F < F_m$ with $F_m$ is the maximum frequency. To do this, apply function fftshift() to the frequency responses and set the frequency $F$ as:

$-(N-1)/2 \leq F \leq (N-1)/2.$

```
Xmagshift = fftshift(Xmag);
Xphaseshift = fftshift(Xphase);
kshift = -(N-1)/2:(N-1)/2;
Fshift = Fs*kshift/N;
```

Below is how the frequency shifted spectrum can be plotted.

```
figure,
subplot(2,1,1), plot(Fshift/1000,Xmagshift)
xlabel('F (kHz)'), ylabel('|X(F)|')
xticks(-5:5)
title('FT Magnitude Response')
grid on

subplot(2,1,2), plot(Fshift/1000,Xphaseshift/pi)
xlabel('F (kHz)'), ylabel('\angle X(F)')
xticks(-5:5)
yticks(-1:1), ytickformat('%g\x3C0')
title('FT Phase Response')
grid on
```



FT Magnitude Response
6000
4000
2000
0
|X(F)|
-5
-4
-3
-2
-1
0
1
2
3
4
5
F (kHz)
FT Phase Response
1π
0π
-1π
∠ X(F)
-5
-4
-3
-2
-1
0
1
2
3
4
5
F (kHz)

# 2.0 DTFT with Different Frequency Sampling Fs

## 2.1 Introduction

For discrete-time, there are two types of frequency, namely angular frequency ($\omega$) and normalized frequency ($f$). Both are dependent on the value of frequency sampling $F_s$ such that:

$$\omega = \frac{2\pi F}{F_s}$$

$$f = F/Fs$$

Thus, one signal with different $F_s$ will have a different DTFT plot. To demonstrate this, let's consider the following signal:

$$x(t) = \ sin(2\pi(1000)t) \quad for\ \ 0 \leq t < 1\ s$$

We will plot the DTFT of $x(t)$ for $F_s$ equals to 10kHz, 5kHz, 4kHz and 3kHz.

## 2.2 The Frequency Response

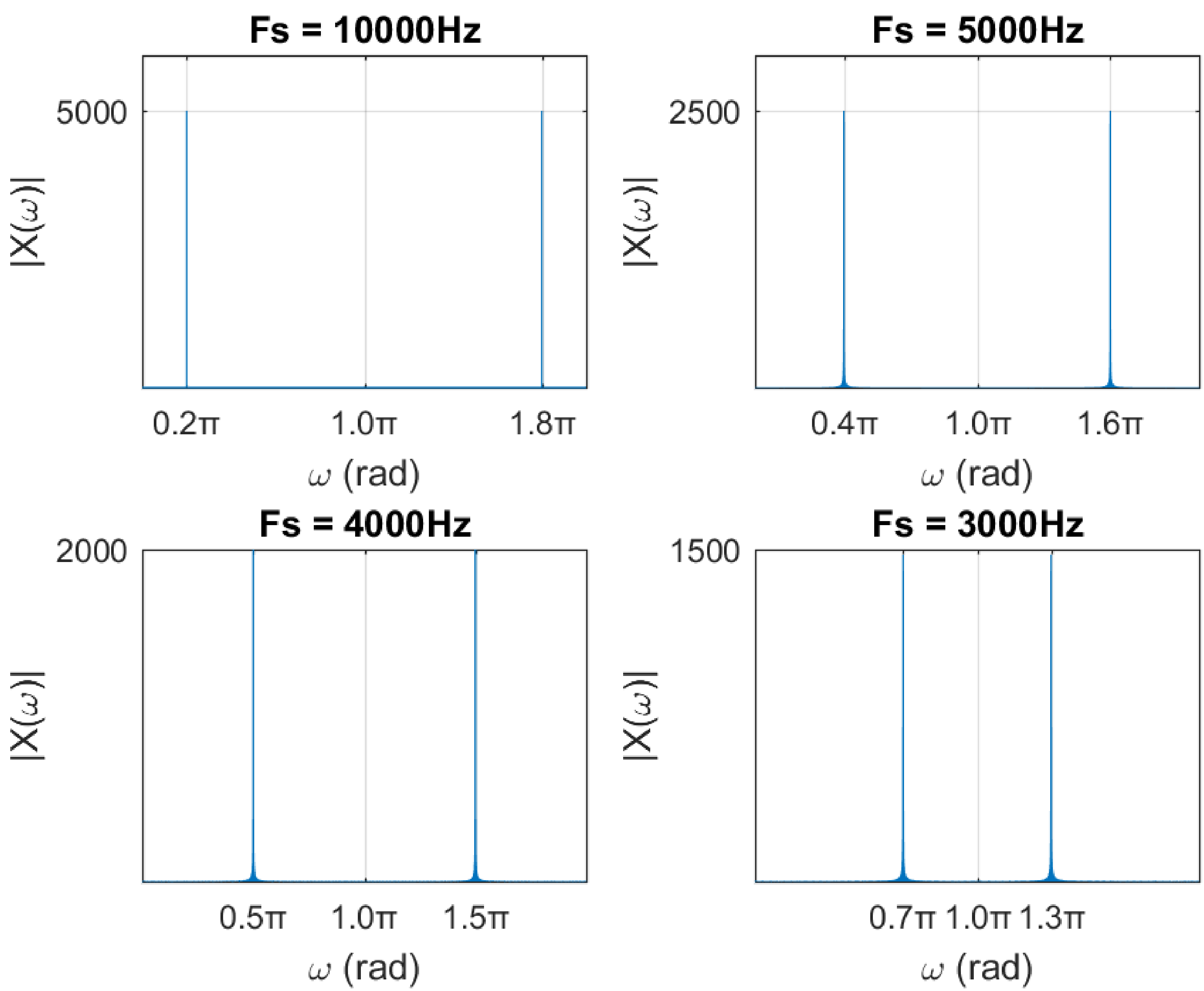
By using fft() function, below is the code to obtain the DTFT for $F_s$ equals to 10kHz, 5kHz, 4kHz and 3kHz. Note that only the magnitude response, $|X(\omega)|$ will be considered in this example.

```
Fs = [10000 5000 4000 3000];
n1 = 0:1*Fs(1)-1; n2 = 0:1*Fs(2)-1; n3 = 0:1*Fs(3)-1; n4 =
0:1*Fs(4)-1;

x1 = cos(2*pi*1000*n1/Fs(1));
x2 = cos(2*pi*1000*n2/Fs(2));
x3 = cos(2*pi*1000*n3/Fs(3));
x4 = cos(2*pi*1000*n4/Fs(4));

N = length(x1);
X1=fft(x1,N); X2=fft(x2,N); X3=fft(x3,N); X4=fft(x4,N);
Xmag = [abs(X1); abs(X2); abs(X3); abs(X4)];
```

## 2.3 Plotting the Frequency Response

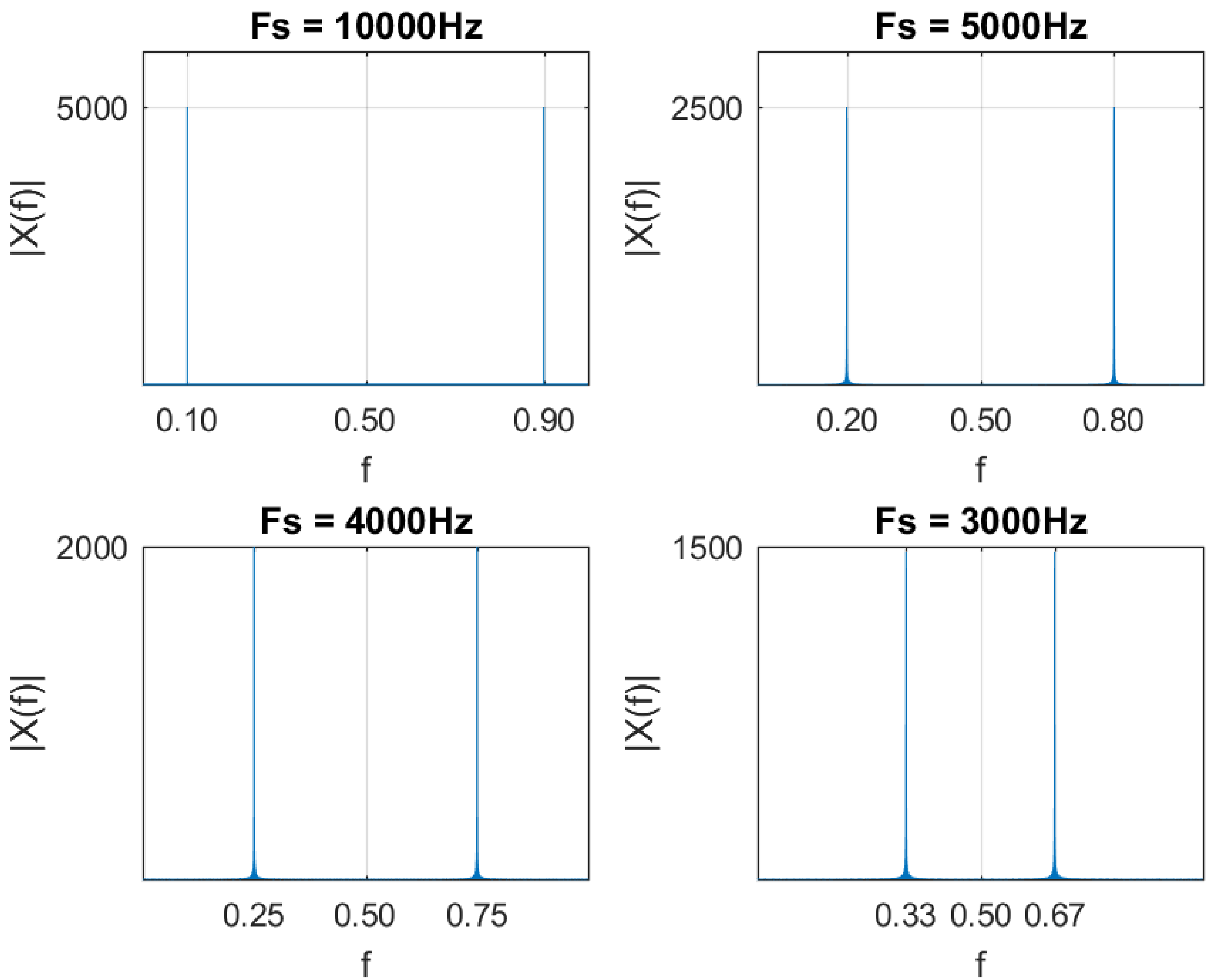
Below is the code to plot the 4 magnitude responses on a single window. Although all the 4 plots look different, they are corresponding to the same continuous-time signal $x(t)$.

```
k = 0:N-1;
w = 2*pi*k/N;

for i = 1:4
    subplot(2,2,i), plot(w/pi,Xmag(i,:))
    xlabel('\omega (rad)'), ylabel('|X(\omega)|')
    xtickformat('%.1f\x3C0')
    xticks([2000/Fs(i) 1 2-2000/Fs(i)]), yticks(Fs(i)/2)
    titlestring = sprintf('Fs = %dHz',Fs(i));
    title(titlestring)
    grid on
end
```


Fs = 10000Hz
5000
|X(ω)|
0.2π
1.0π
1.8π
ω (rad)
Fs = 5000Hz
2500
|X(ω)|
0.4π
1.0π
1.6π
ω (rad)
Fs = 4000Hz
2000
|X(ω)|
0.5π
1.0π
1.5π
ω (rad)
Fs = 3000Hz
1500
|X(ω)|
0.7π 1.0π 1.3π
ω (rad)

And below is the code to plot magnitude response when normalized frequency $f$ is used.

```
figure
for i = 1:4
    subplot(2,2,i), plot(w/(2*pi),Xmag(i,:))
    xlabel('f'), ylabel('|X(f)|')
    xtickformat('%.2f')
    xticks([1000/Fs(i) 0.5 1-1000/Fs(i)]), yticks(Fs(i)/2)
    titlestring = sprintf('Fs = %dHz',Fs(i));
    title(titlestring)
    grid on
end
```


Fs = 10000Hz
5000
|X(f)|
0.10 0.50 0.90
f
Fs = 5000Hz
2500
|X(f)|
0.20 0.50 0.80
f
Fs = 4000Hz
2000
|X(f)|
0.25 0.50 0.75
f
Fs = 3000Hz
1500
|X(f)|
0.33 0.50 0.67
f

# 3.0 Frequency Response of FIR

## 3.1 Introduction

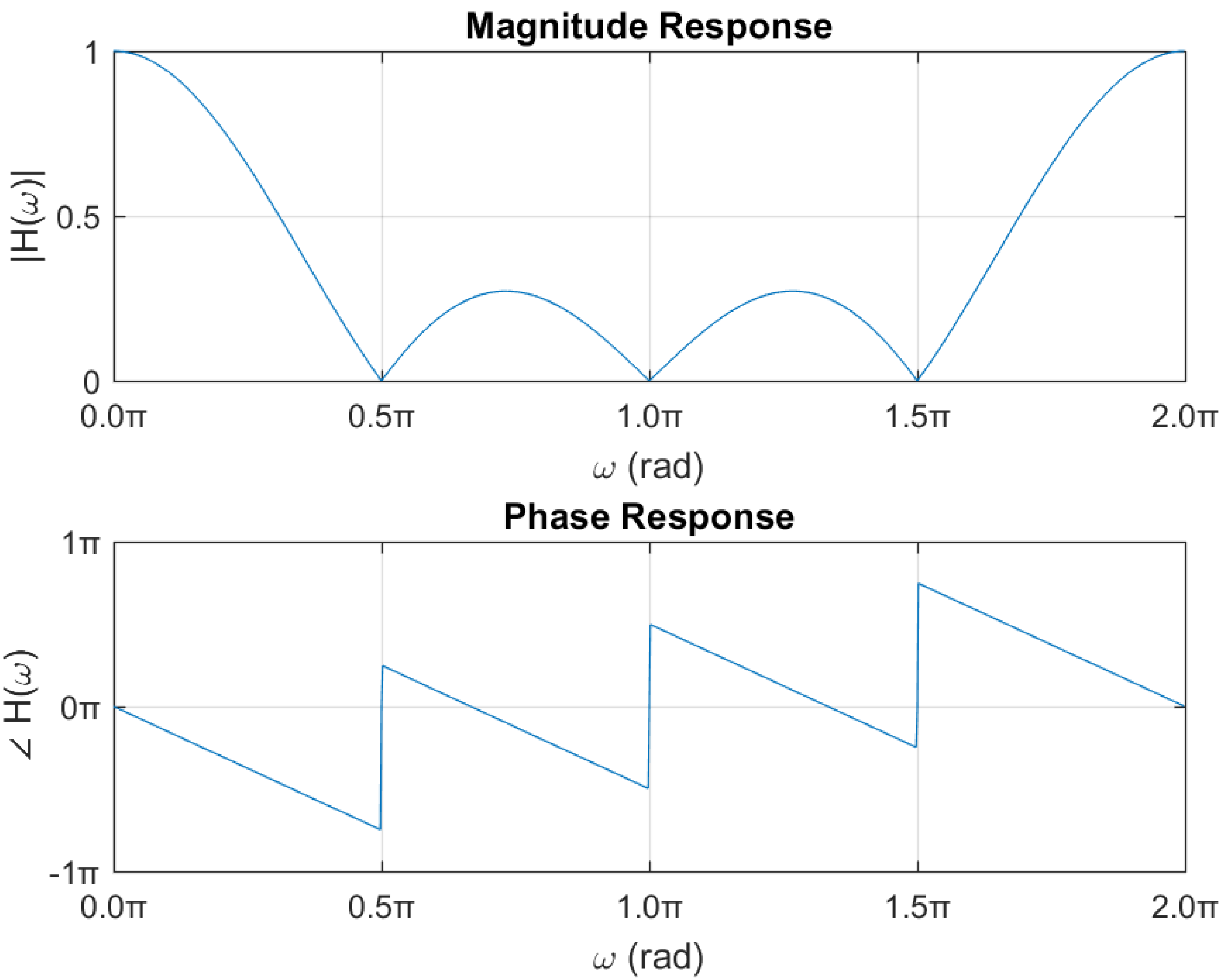
Impulse response of an FIR system is normally consisting of low number of samples. Thus, DTFT of the FIR using fft() function needs a second input to increase the detail of the frequency domain. The second input is $N$, number of samples in frequency domain.

Now, let's consider impulse response $h[n] = [0.25\ 0.25\ 0.25\ 0.25]$. Below is how the second input is applied to the fft() function. Here we set $N = 1000$.

```
h = [0.25 0.25 0.25 0.25];
N = 1000;
H = fft(h,N);
Hmag = abs(H);
Hphase = angle(H);
```

# 3.2 Plotting the Frequency Responses

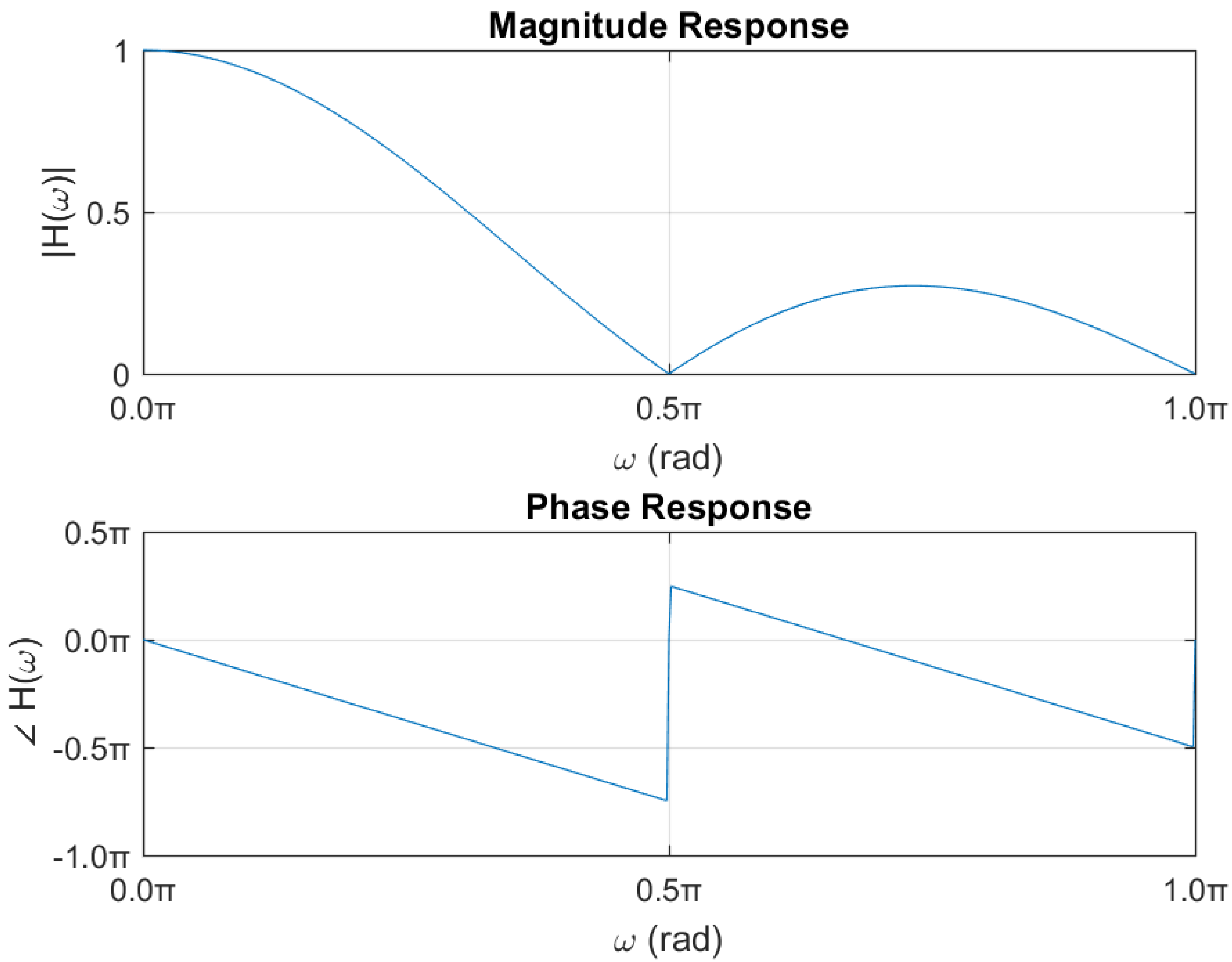
```
k = 0:N-1;
w = 2*pi*k/N;

subplot(2,1,1), plot(w/pi,Hmag)
xlabel('\omega (rad)'), ylabel('|H(\omega)|')
xticks(0:0.5:2), xtickformat('%.1f\x3C0')
title('Magnitude Response')
grid on
subplot(2,1,2), plot(w/pi,Hphase/pi)
xlabel('\omega (rad)'), ylabel('\angle H(\omega)')
xticks(0:0.5:2), xtickformat('%.1f\x3C0')
yticks(-1:1), ytickformat('%d\x3C0')
title('Phase Response')
grid on
```


Magnitude Response
|H(ω)|
1
0.5
0
0.0π
0.5π
1.0π
1.5π
2.0π
ω (rad)
Phase Response
∠ H(ω)
1π
0π
-1π
0.0π
0.5π
1.0π
1.5π
2.0π
ω (rad)

## 3.3 Single-Sided Spectrum

Impulse response is also normally called filter. In this example, the $h[n]$ is a lowpass filter. For filter, it is more appropriate to display only half of the frequency response. The following is the code to plot half of the frequency response. Note that now, the shape of the lowpass filter looks more reasonable.

```
N2 = (N+2)/2;
subplot(2,1,1), plot(w(1:N2)/pi,Hmag(1:N2))
xlabel('\omega (rad)'), ylabel('|H(\omega)|')
xticks(0:0.5:2), xtickformat('%.1f\x3C0')
title('Magnitude Response'), grid on

subplot(2,1,2), plot(w(1:N2)/pi,Hphase(1:N2)/pi)
xlabel('\omega (rad)'), ylabel('\angle H(\omega)')
xticks(0:0.5:2), xtickformat('%.1f\x3C0')
yticks(-1:0.5:1), ytickformat('%.1f\x3C0')
title('Phase Response'), grid on
```


Magnitude Response
1
0.5
0
|H(ω)|
0.0π
0.5π
1.0π
ω (rad)
Phase Response
0.5π
0.0π
-0.5π
-1.0π
∠ H(ω)
0.0π
0.5π
1.0π
ω (rad)

# 4.0 Frequency Response of IIR

# 4.1 Introduction

Since IIR has infinite length, using fft() function to obtain the spectrum (frequency response) is not possible. As an option, we can use freqz() function to plot the IIR spectrum. By default, freqz() return a single-sided spectrum.

We will discuss two syntax of the freqz() as below:

1) `[H,w] = freqz(b,a,N)`
Returns the N-point frequency response vector, H, and the corresponding angular frequency vector, w, for the digital filter with numerator and denominator polynomial coefficients stored in b and a, respectively.

2) `[H,F] = freqz(b,a,N,Fs)`
Returns the frequency response vector, H, and the corresponding physical frequency vector, F, in Hz for the digital filter with numerator and denominator polynomial coefficients stored in b and a, respectively, given the sampling rate, Fs.

# 4.2 Filter Coefficient

Let's consider the following difference equation (DE).

$$y[n] = 0.1x[n] + 0.9y[n-1]$$

Rearrange all the output to the left side of the equation give us,

$$y[n] - 0.9y[n-1] = 0.1x[n]$$

From there, weight on the input sequence $x$ is the coefficient $b$ and weights on the output sequence $y$ are the coefficient $a$. Thus, a = [1, -0.9] and b = 0.1. Then, set the $N$ value.

```
a = [1 -0.9];
b = 0.1;
```

# 4.3 Spectrum based on Angular Frequency

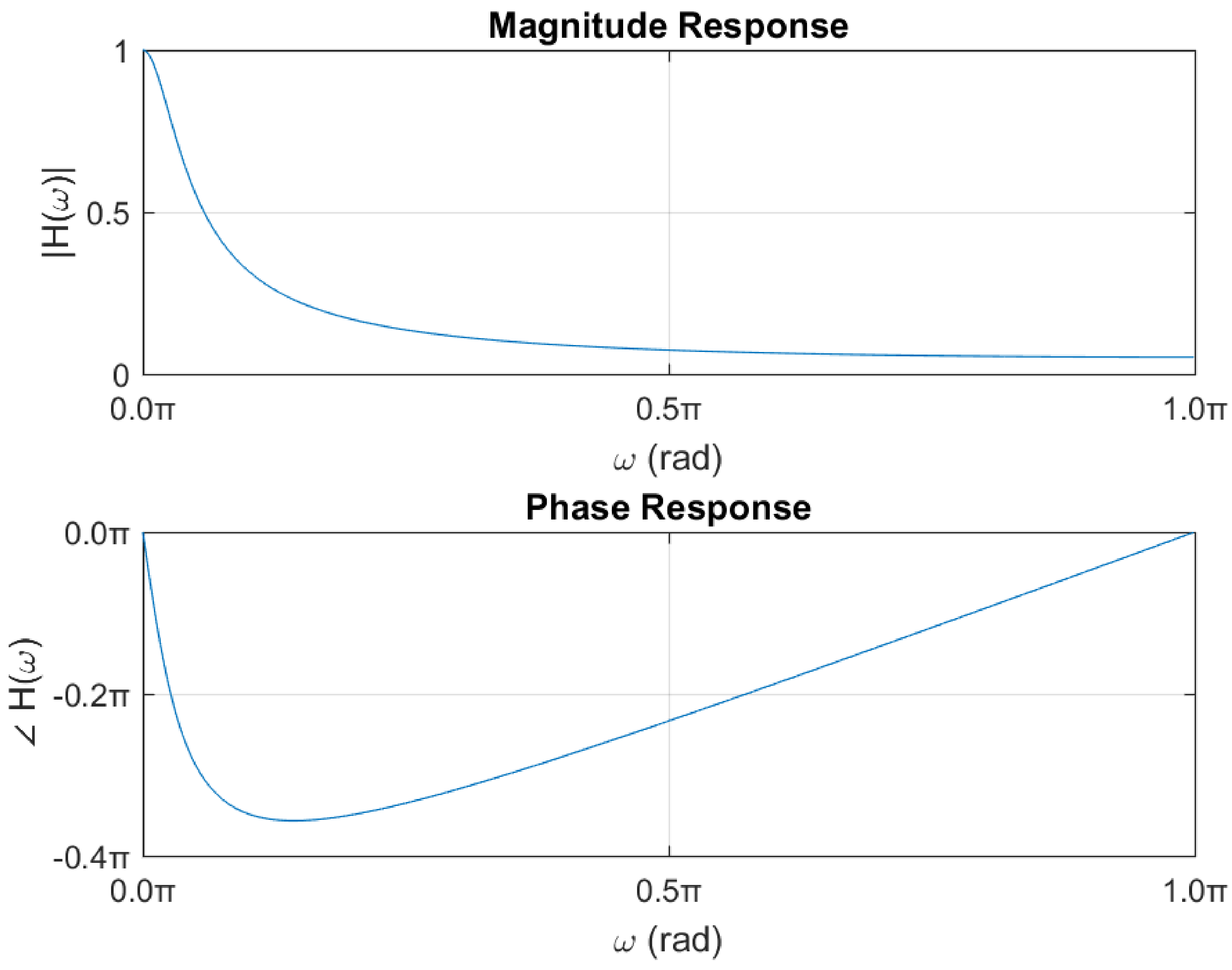
When using the first syntax, it will return the spectrum corresponding angular frequency, $\omega$. Physically, $\omega$ is a continuous value. However, since freqz() function is actually computing DFT, the returned $\omega$ value is actually a sampled angular frequency, which correspond to discrete frequency $k$. Because $\omega$ is a discret samples, $N$ is used to determine the number of its samples. Note that N is similar to the N used in N-point DFT.

In this example, setting $N = 1000$ will be good enough.

```
N = 1000;

[H,w] = freqz(b,a,N);
Hmag = abs(H);
Hphase = angle(H);
```

Below is how plotting the spectrum can be coded.

```
subplot(2,1,1), plot(w/pi,Hmag)
xlabel('\omega (rad)'), ylabel('|H(\omega)|')
xticks(0:0.5:2), xtickformat('%.1f\x3C0')
title('Magnitude Response')
grid on

subplot(2,1,2), plot(w/pi,Hphase/pi)
xlabel('\omega (rad)'), ylabel('\angle H(\omega)')
xticks(0:0.5:2), xtickformat('%.1f\x3C0')
yticks(-1:0.2:1), ytickformat('%.1f\x3C0')
title('Phase Response')
grid on
```


Magnitude Response
1
0.5
0
|H(ω)|
0.0π
0.5π
1.0π
ω (rad)
Phase Response
0.0π
-0.2π
-0.4π
∠ H(ω)
0.0π
0.5π
1.0π
ω (rad)

## 4.4 Spectrum based on Physical Frequency

One of the option in freqz() function is to return the spectrum based on physical frequency $F$ in $Hz$ as below:

```
[H,F] = freqz(b,a,N,Fs)
```

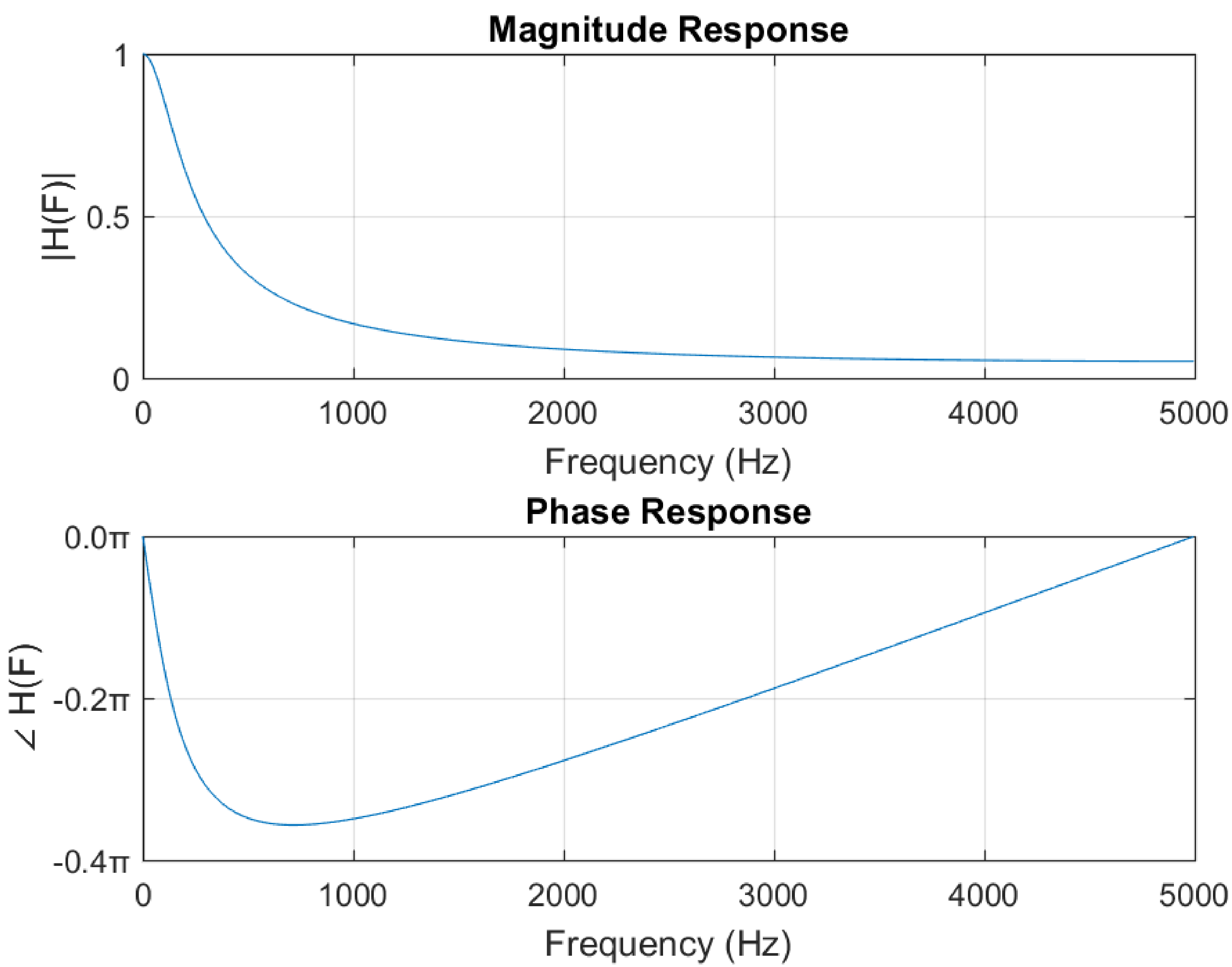
Previously, we know that $N$ is used to sample the angular frequency. Also use as a sampling parameter, $F_s$ is used to sample time. Thus, in this function, $F_s$ is used to map discrete frequency $k$ of DFT to the physical frequency $F$ in $Hz$ such that $F = F_s k/N$ or $F = F_s \omega/2\pi$.

In this example, let's assume the frequency sampling is given as $F_s = 10kHz$.

```
Fs = 10*1e3;

[H,F] = freqz(b,a,N,Fs);
Hmag = abs(H);
Hphase = angle(H);
```

Below is how plotting the spectrum can be coded.

```
figure

subplot(2,1,1), plot(F,Hmag)
xlabel('Frequency (Hz)'), ylabel('|H(F)|')
title('Magnitude Response')
grid on

subplot(2,1,2), plot(F,Hphase/pi)
xlabel('Frequency (Hz)'), ylabel('\angle H(F)')
yticks(-1:0.2:1), ytickformat('%.1f\x3C0')
title('Phase Response')
grid on
```


Magnitude Response
|H(F)|
1
0.5
0
0
1000
2000
3000
4000
5000
Frequency (Hz)
Phase Response
∠ H(F)
0.0π
-0.2π
-0.4π
0
1000
2000
3000
4000
5000
Frequency (Hz)

# 4.5 Function freqz() for FIR

Instead of using fft() function, freqz() function can also be applied to FIR filter. For FIR, coefficient a is always equals to 1 and coefficient b is the impulse response itself. For example, if

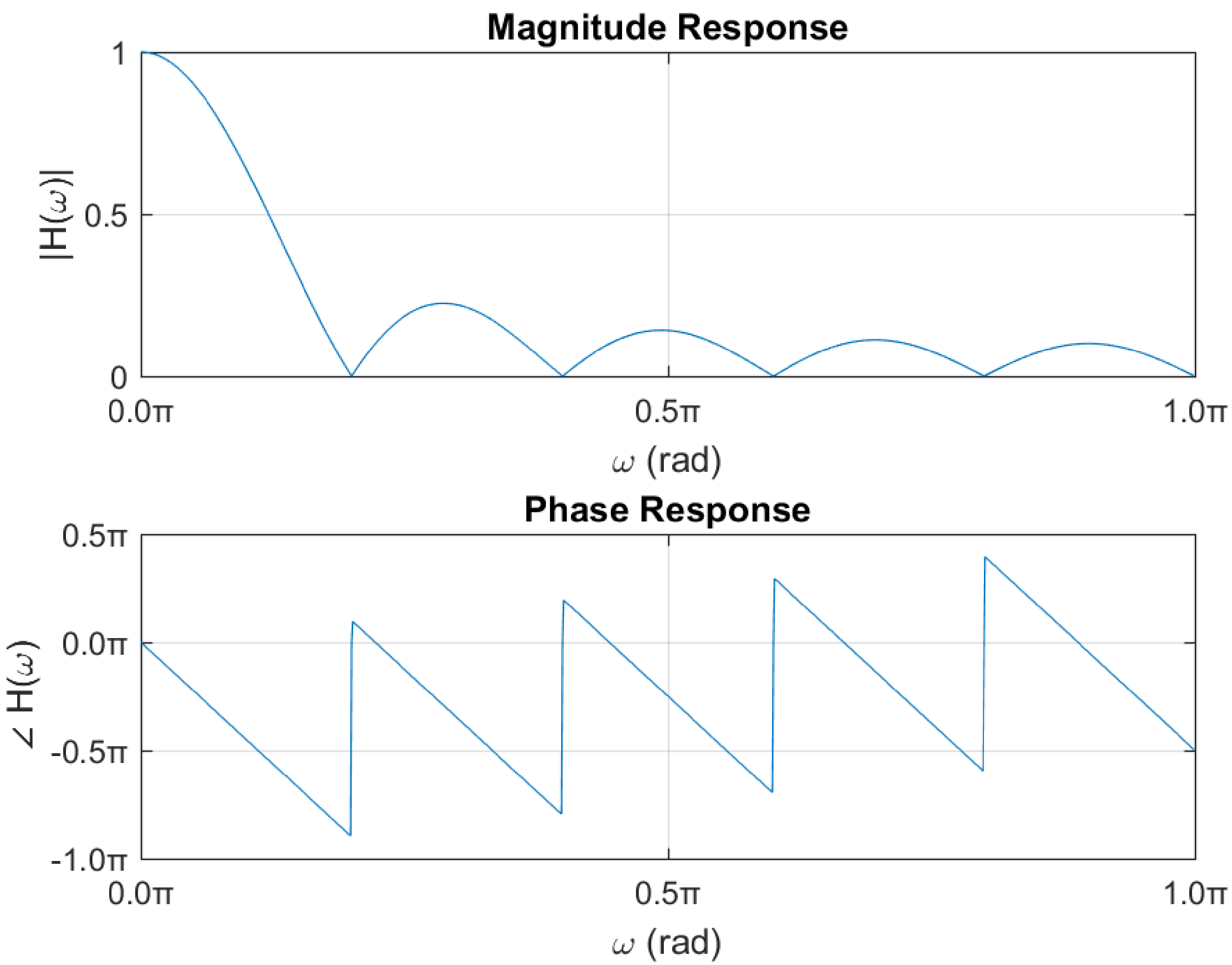
$$h[n] = [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1]$$

Its corresponding DE is:

$$y[n] = 0.1x[n] + 0.1x[n-1] + 0.1x[n-2] + 0.1x[n-3] + 0.1x[n-4] + 0.1x[n-5] + 0.1x[n-6] + 0.1x[n-7] + 0.1x[n-8] + 0.1x[n-9]$$

Thus, $a = 1$ and $b = [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1]$ where $b$ is similar to the impulse response $h[n]$.

```
h = [0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1];
a = 1;
N = 1000;

[H,w] = freqz(h,a,N);
Hmag = abs(H); Hphase = angle(H);

figure
subplot(2,1,1), plot(w/pi,Hmag)
xlabel('\omega (rad)'), ylabel('|H(\omega)|')
xticks(0:0.5:2), xtickformat('%.1f\x3C0')
title('Magnitude Response')
grid on
subplot(2,1,2), plot(w/pi,Hphase/pi)
xlabel('\omega (rad)'), ylabel('\angle H(\omega)')
xticks(0:0.5:2), xtickformat('%.1f\x3C0')
yticks(-1:0.5:1), ytickformat('%.1f\x3C0')
title('Phase Response')
grid on
```


Magnitude Response
1
0.5
0
|H(ω)|
0.0π
0.5π
1.0π
ω (rad)
Phase Response
0.5π
0.0π
-0.5π
-1.0π
∠ H(ω)
0.0π
0.5π
1.0π
ω (rad)

# 5.0 Filtering in Frequency Domain

## 5.1 Introduction

In LTI system, filtering is a convolution process. From its properties, convolution in time-domain corresponds to multiplication in frequency-domain.

$$y[n] = x[n] * h[n] \quad \rightarrow \quad Y(\omega) = X(\omega).H(\omega)$$

To demonstrate the process, let's consider the following input signal $x[n]$ and an FIR filter $h[n]$. Here, $x[n]$ has two main frequencies, which are $\omega = 0.02\pi$ and $\omega = 0.4\pi$.

$$x[n] = 5cos(0.02\pi n) + cos(0.4\pi n) \quad for\ 0 \leq n \leq 200$$

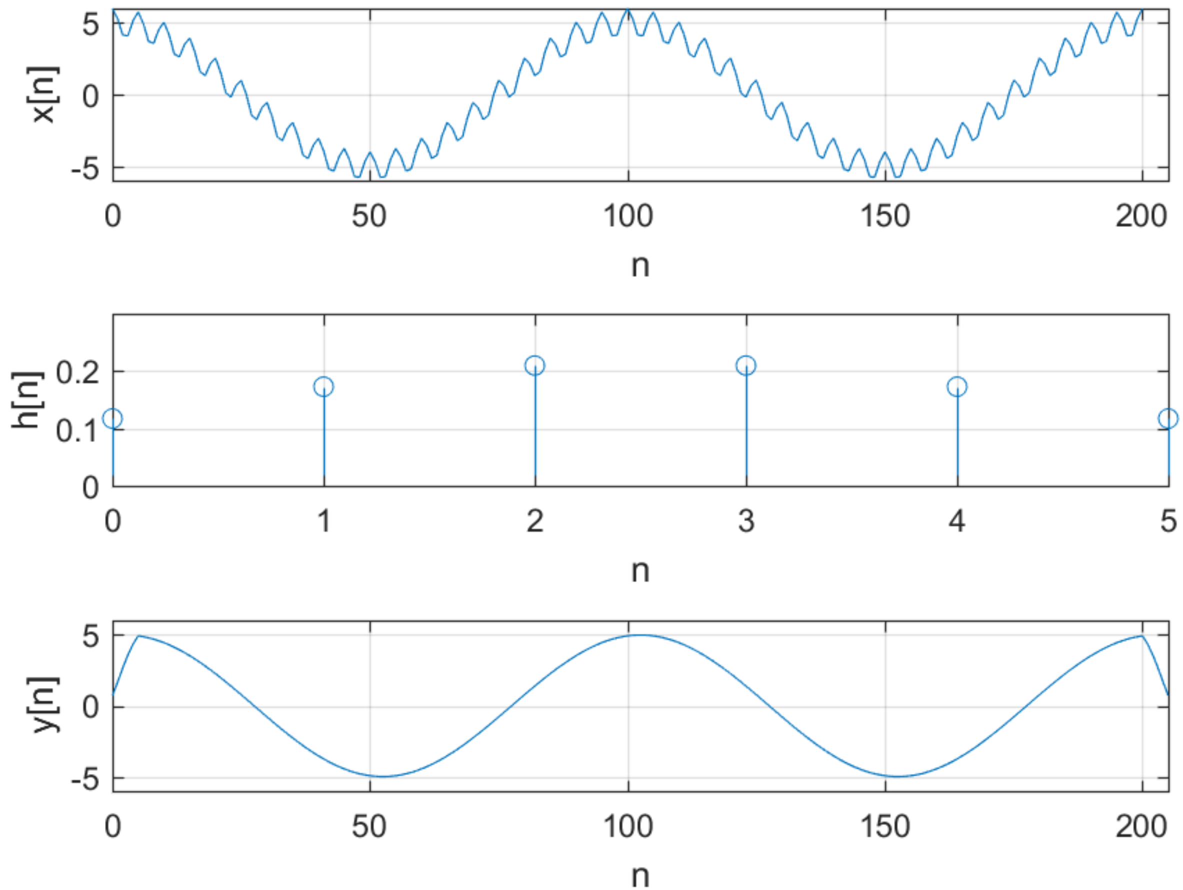
$$h[n] = [0.1174, 0.1729, 0.2098, 0.2098, 0.1729, 0.1174]$$

```
n = 0:200;
x = 5*cos(0.02*pi*n)+cos(0.4*pi*n);
h = [0.1174 0.1729 0.2098 0.2098 0.1729 0.1174];
y = conv(x,h);
```

## 5.2 The Time-Domain Signal

Below is the code on how to plot the signals.

```
subplot(3,1,1), plot(n,x)
xlabel('n'), ylabel('x[n]')
xlim([0 205]), ylim([-6 6]), grid on

subplot(3,1,2), stem(0:5,h)
xlabel('n'), ylabel('h[n]')
xlim([0 5]), ylim([0 0.3]), grid on

subplot(3,1,3), plot(0:205,y)
xlabel('n'), ylabel('y[n]')
xlim([0 205]), ylim([-6 6]), grid on
```




It can be seen from the plots that $h[n]$ filtered the the frequency $\omega = 0.4\pi$ from $x[n]$. At the output, $y[n]$ shows a clean signal with only frequency $\omega = 0.02\pi$ left. To check the frequency on the $y[n]$ plot, convert the angular frequency in $\omega$ to time period $t_p$ where $t_p = 2\pi/\omega$. In this case, $t_p = 2\pi/0.02\pi = 100$.

## 5.3 The Frequency-Domain Spectrum

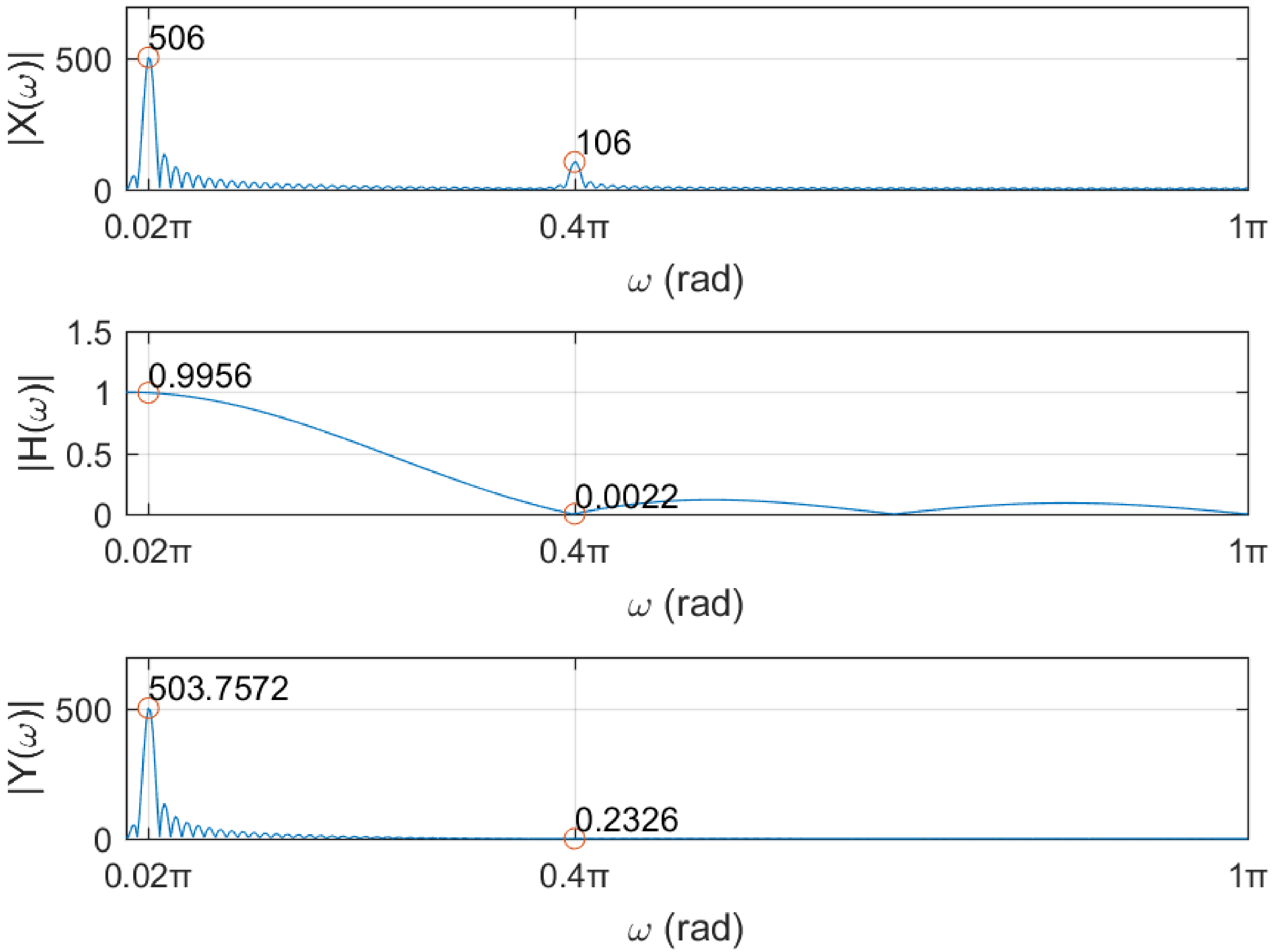
The following is the code to plot the frequency response of the $x[n]$, $h[n]$ and $y[n]$.

```
N=1000;
N2 = (N+2)/2;
k = 0:N2-1;
w = 2*pi*k/N;

% Single-sided Magnitude Respose
X = fft(x,N); Xmag = abs(X(1:N2));
H = fft(h,N); Hmag = abs(H(1:N2));
Y = fft(y,N); Ymag = abs(Y(1:N2));

subplot31(1,w,Xmag,700,80,'|X(\omega)|')
subplot31(2,w,Hmag,1.5,0.15,'|H(\omega)|')
subplot31(3,w,Ymag,700,80,'|Y(\omega)|')
```

## 5.4 User-defined Function to Plot the Spectrum

The function will plot 3 magnitude responses on a single window. Other than the normal plot labelling, the function also marks the $0.02\pi$ and $0.4\pi$ angular frequencies with red circle and shows their magnitude values.

```
function subplot31(num,w1,Mag,Ylim,Yoffset,ystr)
    % Get magnitude value for frequency 0.02pi and 0.4pi
    w2 = [0.02 0.4];
    k2 = w2*1000/2;
    mag = round(Mag(k2+1),4);

    subplot(3,1,num), plot(w1/pi,Mag,w2,mag,' o')
    strx = cellstr(string(mag));
    text(w2,mag+Yoffset,strx)
    xticks([0.02 0.4 1]), xtickformat('%g\x3C0')
    xlabel('\omega (rad)'), ylabel(ystr)
    ylim([0 Ylim])
    grid on
end
```


|X(ω)|
500
0
506
106
0.02π
0.4π
1π
ω (rad)
|H(ω)|
1.5
1
0.5
0
0.9956
0.0022
0.02π
0.4π
1π
ω (rad)
|Y(ω)|
500
0
503.7572
0.2326
0.02π
0.4π
1π
ω (rad)

From the plots, we can observe how the frequency $\omega = 0.4\pi$ in $x[n]$ is filtered by $h[n]$. Again, filtering in time-domain is a convolution process. In frequency-domain, filtering becomes a multiplication process. If we look at the frequency $\omega = 0.4\pi$, we can see a peak on $|X(\omega)|$ with value of 106. At the same frequency, $|H(\omega)| = 0.0022$. To get $|Y(\omega)|$, we multiply $|X(\omega)|$ and $|H(\omega)|$ such that:

$$|Y(0.4\pi)| = |X(0.4\pi)|.|H(0.4\pi)| = 106 \times 0.0022 = 0.2326$$

Compared to $|X(0.4\pi)| = 106$, the $|Y(0.4\pi)| = 0.2326$ is a significant attenuation. Therefore, when we look at the $y[n]$ plot, the frequency $\omega = 0.4\pi$ is barely seen.